hashlock.

Security Audit

Hikari Finance (DeFi)

August 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

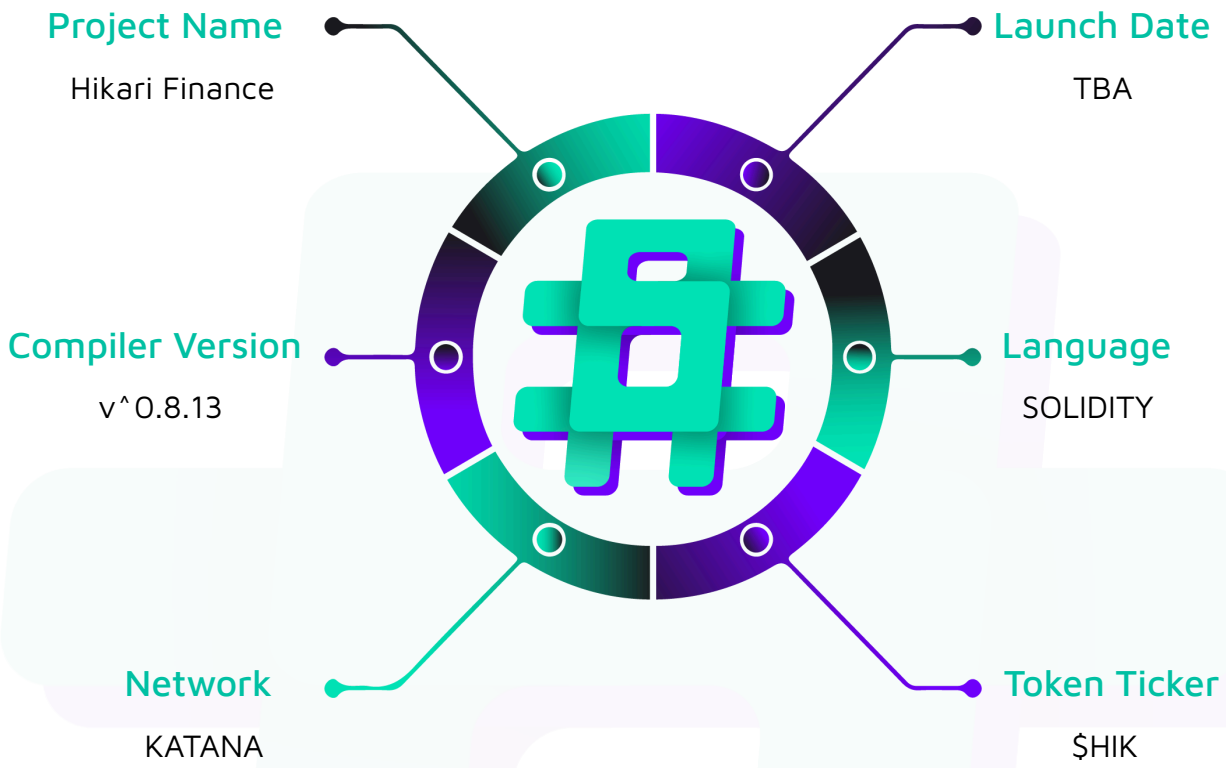
The Hikari Finance team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

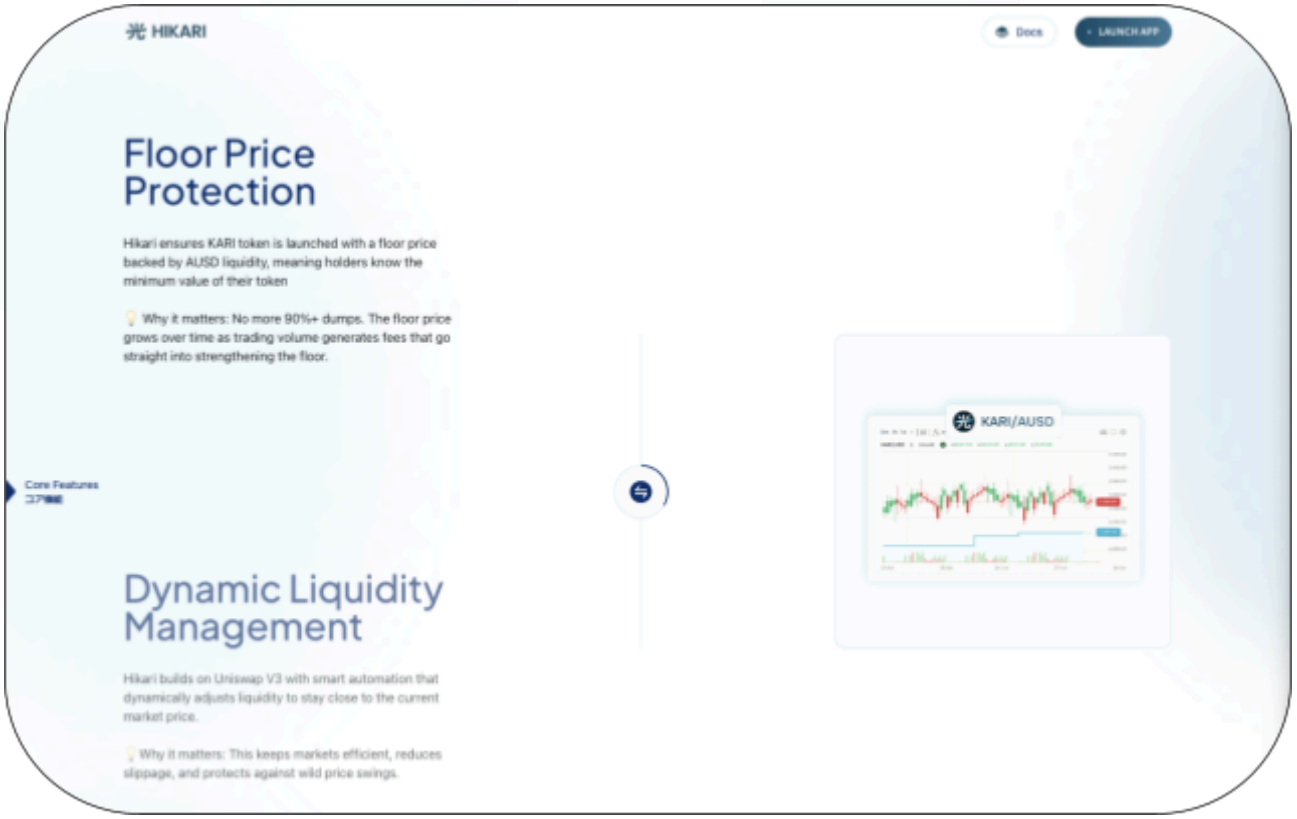
Hikari is a decentralized yield platform powered by $KARI, the only token with a rising, guaranteed floor price. When users stake $KARI, they don't receive more tokens, instead, they unlock the liquidity backing the floor to be deployed across DeFi yield strategies. This Farming as a Service (FaaS) model turns passive backing into active yield, reinforcing $KARI's price floor over time.

Project Name: Hikari Finance
Project Type: DeFi
Compiler Version: ^0.8.13
Website: https://hikari.finance/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Hikari Finance project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Hikari Finance Smart Contracts**
Platform	**Katana / Solidity**
Audit Date	**September, 2025**
Contract 1	BasePriceAUSD.sol
Contract 2	SwapExactInput.sol
Contract 3	KariToken.sol
Audited GitHub Commit Hash	64034ecf00aa2cd9b2b97a66fe0784b7f60341d3
Fix Review GitHub Commit Hash	aa999a4c5fbfbdcd055a07e7c0cb190be578ec59

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

5 Medium severity vulnerabilities

3 Low Severity vulnerabilities

5 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BasePriceAUSD.sol - Allows users to: - Trigger the bonding curve upon price action - Allows admins to: - Initialize pool positions - Adjust ticks	**Contract achieves this functionality.**
SwapExactInput.sol - Allows users to : - Swap one token for another while checking the need for rebalancing simultaneously.	**Contract achieves this functionality.**
KariToken.sol - Standard ERC20 token with minting and burning functionality - Minting is restricted to a specific address	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Hikari Finance project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Hikari Finance project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] BasePriceAUSD#initPoolAndPosition - Pool initialization is vulnerable to frontrunning

Description

The current pool initialization can be frontrun by an attacker to create a pool before and initialize it to an undesired price, causing all the positions to be deployed at different ticks than expected.

Vulnerability Details

The `initPoolAndPosition` function is responsible for creating the initial state for the Hikari protocol. It does so by first deploying the `KARI/AUSD` pool with a selected sqrt price, followed by minting floor, anchor, and discovery position.

However, there can be a case where if the pool is already present and initialized to a specific price, the function will accept that and will proceed to execute further.

```
       function  initPoolAndPosition(uint160  sqrtPriceX96,  int24  floorTick,  uint256
floorAmount, uint256 anchorAmount)
        external onlyOwner
    {
        require(token1 < token0, "Invalid state");
        // uint256 totalToken1Amount = floorAmount + anchorAmount;
        // IERC20(token1).safeTransferFrom(msg.sender, address(this), totalToken1Amount);
        // [1] create and init pool
        pool = nonfungiblePositionManager.createAndInitializePoolIfNecessary(
            token1, token0, UNISWAP_FEE_TIER, sqrtPriceX96
```

```
        );

        //@audit code will accept sqrt price if there is a pool already initialized

        // [2] mint floor

        _ mint Floor(floor mount,floor Tick);

        //calculate the initial discovery token per tick amount

        // [3] mint anchor and discovery

        //_mintAnchorAndDiscovery(anchorAmount);

        _mintAnchor(anchorAmount);

          uint24 TICK_BASE_DIVISION = uint24(ANCHOR_LEFT_LENGTH + ANCHOR_RIGHT_LENGTH) *
(uint24(TICK_SPACING) / uint24(TICK_GAP_LENGTH)); // (20 + 20 ) * (60/30) = 80

        TICK_HEIGHT_BASE = anchorAmount / TICK_BASE_DIVISION;

        TICK_HEIGHT_BASE_KARI = ANCHOR_KARI_balance / TICK_BASE_DIVISION;

       TICK_ZERO = getCurrentTick();

        _mintDiscovery();

        if (IERC20(token1).balanceOf(address(this)) > 0) {

            _increaseLiquidity(floorTokenId, IERC20(token1).balanceOf(address(this)), 0);

            FLOOR_balance += IERC20(token1).balanceOf(address(this));

        }

        // [4] zero approve

        IERC20(token0).approve(address(nonfungiblePositionManager), 0);

        IERC20(token1).approve(address(nonfungiblePositionManager), 0);

    }
```

Let's take a look at `NonFungiblePositionManager`

```
function createAndInitializePoolIfNecessary(
       address token0,
       address token1,
       uint24 fee,
```

```
        uint160 sqrtPriceX96
    ) external payable override returns (address pool) {
        require(token0 < token1);
        pool = IUniswapV3Factory(factory).getPool(token0, token1, fee);

        if (pool == address(0)) {
            pool = IUniswapV3Factory(factory).createPool(token0, token1, fee);
            IUniswapV3Pool(pool).initialize(sqrtPriceX96);
        } else {
            (uint160 sqrtPriceX96Existing, , , , , , ) = IUniswapV3Pool(pool).slot0();
            if (sqrtPriceX96Existing == 0) {
                IUniswapV3Pool(pool).initialize(sqrtPriceX96);}
        }
    }
```

Looking at the code, we can clearly see how the function does nothing if the pool that we are trying to initialize already exists and simply returns the `sqrtPrice`.

An attacker can take advantage of this behaviour by frontrunning `initPoolAndPosition` and deploy the same pool with a skewed `sqrtPrice`.

Proof of Concept

Paste the following `POC` in `BasePriceAUSD.t.sol`

```
   function test_InitAndPoolTest() public {

        vm.startPrank(attacker);

                                                    address       attackerPool       =
INonfungiblePositionManager(address(0xC36442b4a4522E871399CD717aBDD847Ab11FE88)).createAn
dInitializePoolIfNecessary(

                AUSD,

                address(kariToken),

                3000,

                2588040767327182223822189771149429l4

            );

        vm.stopPrank();
```

```
        //uint160 sqrtPriceX96 = PriceMath.priceToSqrtPriceX96(1000, 6);

        uint160 sqrtPriceX96 = 792281625142643375935439503360000000000; // 414486

        console.log(sqrtPriceX96);

        uint160 floorSqrtPriceX96 = 811847080561112494170645202248188555517; // 414973

        int24 unnormalizedFloorTick = TickMath.getTickAtSqrtRatio(floorSqrtPriceX96);

                int24 floorTick = unnormalizedFloorTick / basePrice.TICK_SPACING() *
basePrice.TICK_SPACING();

        console.log(floorTick);

        deal(AUSD, address(this), 1000e6);

        IERC20(AUSD).transfer(address(proxy), 1000e6);

        BasePriceAUSD(address(proxy)).initPoolAndPosition(sqrtPriceX96, floorTick, 900e6,
100e6);

        (, int24 currentTick,,,,,) = IUniswapV3Pool(attackerPool).slot0();

        console.log("currentTick", currentTick);

    }
```

Looking at the logs, we can clearly see how the final `currentTick` is what the attacker initialized.

Impact

Positions will be minted relative to the wrong `sqrt` price, causing the entire model to break.

Recommendation

If the pool exists beforehand, check if the `sqrt` price is well within the expected range or not. If not, manual intervention is required to make it right.

Status

Resolved

[H-02] BasePriceAUSD#uniswapV3SwapCallback - Unrestricted callback will result in attacker stealing all the funds.

Description

The `uniswapV3SwapCallback` function lacks validation of the caller, meaning it does not check if the call actually comes from a legitimate `Uniswap` pool. As a result, any attacker can invoke the function directly and drain all funds from the contract.

Vulnerability Details

The `uniswapV3SwapCallback` is public and callable by anyone. As a result, anybody can call this address and steal all the funds that are present in the contract.

```
   function uniswapV3SwapCallback(
    int256 amount0Delta,
    int256 amount1Delta,
    bytes calldata data
) external override {
    (bool zeroForOne, address _token0, address _token1) = abi.decode(data, (bool,
address, address));
    // Verify only the expected pool is allowed to call back
    require(zeroForOne == false, "Invalid State");
    if (amount0Delta > 0) {
        IERC20(_token0).transfer(msg.sender, uint256(amount0Delta));
    } else if (amount1Delta > 0) {
        IERC20(_token1).transfer(msg.sender, uint256(amount1Delta)); }}
```

Impact

Stealing all the funds present in the contract.

Recommendation

It is recommended to check if the call is coming from the expected pool and then transfer funds.

Status

Resolved

[H-03] BasePriceAUSD#slide - slide will not be triggered under certain conditions.

Description

The function `slide()` is being called every time the tick increases. This is done to make sure that anchor liquidity will be placed closer to the current tick in the case when tick increases.
However, since the condition `anchorTickUpper < floorTickLower` is invalid, the function won't trigger.

Vulnerability Details

During pool initialization, when anchor position is minted, the function explicitly takes care of the condition where `anchorTickUpper > floorTickLower`. In, that case both are considered as equal.

```
   function _mintAnchor(uint256 anchorAmount) internal {
        _maxMint();
        (uint160 sqrtPriceX96, int24 currentTick, uint128 desiredLiquidity,,,,) =
IUniswapV3Pool(pool).slot0();
        snapshotMarketTick = currentTick;
        uint256 balance = IERC20(token0).balanceOf(address(this));
        int24 liquidityTick = (currentTick / TICK_SPACING) * TICK_SPACING;
        anchorTickLower = liquidityTick - (ANCHOR_LEFT_LENGTH * TICK_SPACING);
        anchorTickUpper = liquidityTick + (ANCHOR_RIGHT_LENGTH * TICK_SPACING);

        if (anchorTickUpper > floorTickLower) anchorTickUpper = floorTickUpper;
//@audit can be equal

        uint160 sqrtPriceUpper = TickMath.getSqrtRatioAtTick(anchorTickUpper);
        uint160 sqrtPriceLower = TickMath.getSqrtRatioAtTick(anchorTickLower);
        // more code ..
```

For example, if your input floor tick is `414960`, then the floor tick range is `[414960 - 414900]` , then our calculated tick for the anchor

Will be `anchorTickUpper = liquidityTick + (ANCHOR_RIGHT_LENGTH * TICK_SPACING);`

`i.e. 414960 + (20 * 60 ) = 416160`

Now the function will assign this value equal to `floorTickUpper` `i.e.` `414960`.

However, when `slide()` is called, the function checks if `anchorTickUpper< floorTickLower i.e. 414960 < 414900` , which can never be true.

```
  function slide() external {
        (, int24 currentTick,,,,,) = IUniswapV3Pool(pool).slot0();
        int24 triggerTick = snapshotMarketTick + ANCHOR_TRIGGERED_TICK_LENGTH;
        //if (currentTick < triggerTick) revert NotEnoughPriceChange(currentTick,
triggerTick);
    if (currentTick >= triggerTick && anchorTickUpper < floorTickLower) { // @audit
never be true
```

As a result, the code skips the execution part.

Proof of Concept

Paste the following `POC` in `BasePriceAUSD.t.sol`

```
   function test_Swap_init_kari() public {
        test_InitAndPoolTest();

        SwapExactInput.SwapExactInputParams memory params =
SwapExactInput.SwapExactInputParams({
            token0: AUSD,
            token1: address(kariToken),
            zeroForOne: false,
            amountIn: 200000000e18,
            amountOut: 0,
            deadline: block.timestamp + 20 minutes,
            pool: basePrice.pool(),
            sqrtPriceLimitX96: 0
        });
```

```
        deal(address(kariToken), alice, params.amountIn);
        vm.startPrank(alice);
        IERC20(AUSD).approve(address(swapCallback), params.amountIn);
        kariToken.approve(address(swapCallback), params.amountIn);
        swapCallback.swapExactInput(params);
        vm.stopPrank();
    }
```

Do note that `test_InitAndPool` is nothing but `test_InitAndPoolTest`

Upon looking at the logs, we can clearly see how slide() never actually executed as `console.log("slide");` was not traced.

It is worth noting that , if due to trading activity the price of the pool goes down and `sweep()` is called, it will move the anchor tick range down while not moving the floor tick range, which would be above the anchor tick range.

So, in that case, next time the price moves up, the `slide()` function will execute the rebalance because now the condition `anchorTickUpper < floorTickLower` will be true.

Impact

`slide()` won't trigger when it needs to be, resulting in no cushion left for liquidity absorption.

Recommendation

The current code assumes `anchorTickUpper` to be just below `FloorTickLower` and pattern matches with `anchorTickLower = DiscoveryTickUpper` means, change the assignment of `anchorTickLower to FloorTickLower` instead and consider changing `'<' to '<='`

Status

Resolved

[H-04] BasePriceAUSD - Lack of access control in critical functions will result in liquidity drainage.

Description

Several functions in the contract `_increaseLiquidity()`, `_increaseLiquidityFromManager ()`, `_increaseLiquidityFromManager2()` are declared as public and lack any from of access control.

These functions directly interact with `UniV3's NonFungiblePositionManager` and can move protocol-owned assets into liquidity positions. Because they are callable via any `tokenId`, an attacker can manipulate the contract's liquidity management logic.

Vulnerability Details

An attacker can supply their own tokenId, call this function, and have the protocol deposit its tokens into attacker-owned `NFT` positions.

```
   function _increaseLiquidityFromManager(uint256 tokenId, uint256 amount1Desired,
uint256 amount0Desired) public {
        _collect(tokenId);
        uint256 amountToMint;
        if (IERC20(token0).balanceOf(address(this)) < amount0Desired) {
            amountToMint = amount0Desired - IERC20(token0).balanceOf(address(this));
            KariToken(token0).mint(address(this), amountToMint);
        }
        IERC20(token1).safeTransferFrom(msg.sender, address(this), amount1Desired);
        IERC20(token0).approve(address(nonfungiblePositionManager), amount0Desired);
        IERC20(token1).approve(address(nonfungiblePositionManager), amount1Desired);
        nonfungiblePositionManager.increaseLiquidity(
            INonfungiblePositionManager.IncreaseLiquidityParams({
                tokenId: tokenId,
                amount0Desired: amount1Desired,
                amount1Desired: amount0Desired,
                amount0Min: 0,
                amount1Min: 0,
                deadline: block.timestamp
            })
        );
```

```
    }

    function _increaseLiquidityFromManager2(uint256 tokenId) public {
        _collect(tokenId);
        IERC20(token1).approve(address(nonfungiblePositionManager),
IERC20(token1).balanceOf(address(this)));
        nonfungiblePositionManager.increaseLiquidity(
            INonfungiblePositionManager.IncreaseLiquidityParams({
                tokenId: tokenId,
                amount0Desired: IERC20(token1).balanceOf(address(this)),
                amount1Desired: 0,
                amount0Min: 0,
                amount1Min: 0,
                deadline: block.timestamp + 1 minutes
            })
        );
    }

    function _increaseLiquidity(uint256 tokenId, uint256 amount1Desired, uint256
amount0Desired) public {
        IERC20(token0).approve(address(nonfungiblePositionManager), amount0Desired);
        IERC20(token1).approve(address(nonfungiblePositionManager), amount1Desired);
        nonfungiblePositionManager.increaseLiquidity(
            INonfungiblePositionManager.IncreaseLiquidityParams({
                tokenId: tokenId,
                amount0Desired: amount1Desired,
                amount1Desired: amount0Desired,
                amount0Min: 0,
                amount1Min: 0,
                deadline: block.timestamp
            })
        );
    }
```

`_increaseLiquidity`: Allows adding liquidity into any `NFT` tokenId using the contract's balances.

An attacker can supply their own tokenId, call this function, and have the protocol deposit its tokens into attacker-owned `NFT` positions.

`_increaseLiquidityFromManager`

An attacker can mint an `NFT` with almost zero liquidity, transfer it, then call this function to force the protocol to deposit `AUSD/KARI` into that `NFT`. The tokens are effectively stolen or locked.

`_increaseLiquidityFromManager2`

Same as above, but focused on `AUSD` contribution.

An attacker can direct protocol `AUSD` into attacker-owned positions, draining the treasury.

Impact

Liquidity provision mismanagement and theft of funds

Recommendation

Consider marking them as internal, or if they are being called via an external call, consider allowing only those contracts to access it.

Status

Resolved

Medium

[M-01] BasePriceAUSD - Usage of slot0 for price fetching is not recommended

Description

The core functionalities of the contract, like `sweep()`, `slide()`, and `drop()`, are dependent on fetching `currentTick` via `slot0`. However, it is important to remember that `slot0` only stores the current tick, which can be manipulated.

Vulnerability Details

The current mechanism of triggering the above function based on the price action is dependent on `slot0`.

However, the usage of `slot0` tick is vulnerable to manipulation. An attacker can take a flash loan and manipulate the current tick and can trigger the above functions without any need.
This might open up arbitrage opportunities.

```
 function sweep() external {

        uint256 bondingCurveAmount;

        uint256 bondingCurveMultiplier;

        (, int24 currentTick,,,,,) = IUniswapV3Pool(pool).slot0();

// more code...

  function slide() external {

        (, int24 currentTick,,,,,) = IUniswapV3Pool(pool).slot0();

// more code...

  function drop() external {

        if (block.timestamp >= dropTimestamp + 1 days){ // +1 days

        (, int24 currentTick,,,,,) = IUniswapV3Pool(pool).slot0();

// more code...
```

Impact

An attacker can push the tick to any value to trigger protocol actions, then restore the price atomically.

Recommendation

It is recommended to time weighted average price instead.

Status

Resolved

[M-02] BasePriceAUSD - Usage of block.timestamp does not equate to instantaneous execution.

Description

Submitting deadline as `block.timestamp` is a bad idea since a malicious miner can hold it until the transaction is favourable to them.

Vulnerability Details

Using `block.timestamp` for expiry doesn't ensure immediate execution and allows miners to delay transactions for their benefit, causing slippage or liquidations. To prevent this, expiry times should be set off-chain and explicitly defined by the caller to reduce `MEV` risk.

There are multiple instances of this issue:

1. `_mintFloor`
2. `_mintAnchor`
3. `_mintDiscovery`
4. `_increaseLiquidityFromManager`
5. `_increaseLiquidityFromManager2`
6. `_increaseLiquidity`
7. `_decreaseLiquidity`

Impact

Allows `MEV` to be extracted by delaying tx inclusion.

Recommendation

Provide a custom deadline parameter instead of just `block.timestamp`. Also, use slippage protection wherever necessary.

Status

Resolved

[M-03] BasePriceAUSD#slide - Previous intermediary positions become inaccessible

Description

The `slide()` function calls `_mintIntermediaryToDiscovery()` without checking if `intermediaryUpperTokenId` is already set, causing previous positions to become inaccessible.

Vulnerability Details

In the `slide()` function, the code calls `_mintIntermediaryToDiscovery()` to mint a new intermediary position. The `_mintIntermediaryToDiscovery()` function directly assigns a new token ID to `intermediaryUpperTokenId` without emptying the previous position. This means if `slide()` is called multiple times, the previous `intermediaryUpperTokenId` position will be overwritten and lost from tracking, but the position remains in the contract's custody.

Impact

Previous intermediary positions become inaccessible and their assets cannot be recovered through normal protocol operations, effectively locking funds in the contract. While the contract is upgradeable and the team could potentially recover these positions through upgrades, this represents a temporary loss of access to protocol assets.

Recommendation

Add a check in `slide()` to empty the existing `intermediaryUpperTokenId` before minting a new one.

Status

Resolved

[M-04] BasePriceAUSD#drop - Missing zero check before emptying intermediary liquidity

Description

The `drop()` function calls `_emptyLiquidity(intermediaryUpperTokenId)` without verifying if the token ID is set, which can cause transaction reverts when the position doesn't exist.

Vulnerability Details

In the `drop()` function, the code directly calls `_emptyLiquidity(intermediaryUpperTokenId)` without checking if `intermediaryUpperTokenId` is non zero. The `intermediaryUpperTokenId` is only set when a new position is minted in certain cases and may remain `0` if no intermediary position was created or if was already emptied. When `_emptyLiquidity()` is called with a `0` token `ID`, it will revert as it cannot empty the liquidity of a position that does not exist. This lack of validation is inconsistent with other parts of the codebase properly checking for zero values before calling `_emptyLiquidity()`.

Impact

This can cause the `drop()` function to revert when `intermediaryUpperTokenId` is `0`, potentially disrupting operations that depend on this function like in the `checkRebalance()` function in the `SwapExactInput` contract, reverting a legit swap of a user.

Recommendation

Add a zero check before calling `_emptyLiquidity(intermediaryUpperTokenId)` similar to the pattern used elsewhere in the codebase.

Status

Resolved

[M-05] BasePriceAUSD#sweep - Integer underflow in furtherTick calculation causes DoS

Description

The `sweep()` function calculates `furtherTick` using unsafe subtraction that can underflow when `currentTick` is less than `discoveryTickLower`, causing a denial of service.

Vulnerability Details

In the `sweep()` function, the code calculates:

```
uint24 furtherTick = uint24(currentTick) - uint24(discoveryTickLower);
```

This subtraction can underflow if `currentTick < discoveryTickLower`. While the protocol provides liquidity up to `discoveryTickLower`, external users can provide liquidity at lower ticks.

When price moves below `discoveryTickLower` due to external liquidity, the `furtherTick` calculation will underflow, causing the transaction to revert and preventing the protocol from rebalancing.

Proof of Concept

Paste the following `POC` in `BasePriceAUSD.t.sol`:

```
function test_sweep_below_discovery_tick_lower() public {

swapCallback = new SwapExactInput(address(basePrice), nonfungiblePositionManagerKatana, address(this));

      test_InitAndPool();

INonfungiblePositionManager.MintParams memory mintBelowDiscoveryParams = INonfungiblePositionManager.MintParams({

        token0: AUSD,

        token1: address(kariToken),

        fee: 3000,
```

```
        tickLower: basePrice.discoveryTickLower() - 60,
        tickUpper: basePrice.discoveryTickLower() + 60,
        amount0Desired: 0,
        amount1Desired: 2508458972293228973635986O,
        amount0Min: 0,
        amount1Min: 0,
        recipient: address(alice),
        deadline: block.timestamp
    });
vm.prank(address(basePrice));
KariToken(kariToken).mint(alice, mintBelowDiscoveryParams.amount1Desired);
vm.startPrank(alice);
IERC20(kariToken).approve(address(nonfungiblePositionManagerKatana),
mintBelowDiscoveryParams.amount1Desired);
(uint256                          userLiquidityPositionId,,,)                          =
INonfungiblePositionManager(nonfungiblePositionManagerKatana).mint(mintBelowDiscoveryPara
ms);
SwapExactInput.SwapExactInputParams memory params = SwapExactInput.SwapExactInputParams({
        token0: AUSD,
        token1: address(kariToken),
        zeroForOne: true,
        amountIn: 11000000e6,
        amountOut: 0,
        deadline: block.timestamp,
        pool: basePrice.pool(),
        sqrtPriceLimitX96: 0
    });
```

```
deal(AUSD, alice, params.amountIn);

vm.startPrank(alice);

IERC20(AUSD).approve(address(swapCallback), params.amountIn);

swapCallback.swapExactInput(params); // this will trigger sweep()

vm.stopPrank();

}
```

Impact

The protocol becomes unable to rebalance using the `sweep()` function when external users provide liquidity below the `discoveryTickLower` tick, resulting in a denial of service.

Recommendation

Add a check to ensure the subtraction is only performed when `currentTick >= discoveryTickLower`.

Status

Resolved

Low

[L-01] SwapExactInput#checkSolvencyInvariant - Incorrect circulating supply calculation

Description

The `checkSolvencyInvariant()` function calculates circulating supply using total supply without subtracting KARI tokens held in the pool and manager contract, leading to inaccurate solvency calculations.

Vulnerability Details

The code calculates `circulatingSupply` as `IERC20(token0).totalSupply()`. However, this includes `KARI` tokens that are locked in the `Uniswap` pool and held by the manager contract, which should not be considered as circulating supply. The function then compares this inflated circulating supply against the protocol's capacity to determine solvency status.

Impact

This can lead to incorrect solvency assessments if the function is used for off-chain monitoring.

Recommendation

Calculate the actual circulating supply by subtracting `KARI` tokens held in the pool and manager contract from the total supply.

Status

Resolved

[L-02] SwapExactInput#uniswapV3SwapCallback - Callback validation can be bypassed

Description

The `uniswapV3SwapCallback()` function can be called by anyone with manipulated data to bypass the pool address validation, potentially allowing unauthorized token transfers.

Vulnerability Details

The code decodes the `expectedPool` address from the callback `data` and validates it against `msg.sender`. However, since the `data` parameter is controlled by the user, an attacker can call this function directly and pass their own address as the `expectedPool` in the encoded data bypassing the validation check.

The function then transfers tokens to `msg.sender` based on the `amount0Delta` and `amount1Delta` parameters, which are also controlled by the caller.

Impact

This vulnerability could allow an attacker to drain any tokens held by the `SwapExactInput` contract. However, the impact is limited as this contract is not designed to store tokens long term, just temporarily to perform a swap.

Recommendation

Consider storing the pool address temporarily in a contract variable when `swapExactInput()` is called and validating it against `msg.sender` instead of decoding it from the callback data in `uniswapV3SwapCallback()`.

Status

Resolved

[L-03] SwapExactInput - Manager variable lacks proper visibility and event tracking

Description

The `manager` variable is `internal`, and the `setManager()` function does not emit an event when the manager address is changed, making it difficult to track changes to this variable used in solvency calculations.

Vulnerability Details

The `manager` variable is not `public` and used in the `checkSolvencyInvariant()` function to calculate the protocol's capacity. The `setManager()` function allows the admin to update this address but does not emit any event to notify about the change.

Impact

The internal visibility and absence of event emission make it difficult to track changes to the manager address, potentially affecting external monitoring systems and reducing transparency for users who need to know the current manager address for solvency calculations.

Recommendation

Consider making the `manager` variable `public` and emit an event in the `setManager()` function.

Status

Resolved

QA

[Q-01] BasePriceAUSD - Consider removing unused variables

Description

The `desiredLiquidity` and `sqrtPriceX96` are some of the variables that are not being used in the function execution of `_mintAnchor`.

Recommendation

Consider removing unused variables.

Status

Resolved

[Q-02] BasePriceAUSD - Empty natspec

Description

The presence of empty natspec over critical functions is not recommended.

Recommendation

It is recommended to adhere to proper `natspec` instead.

Status

Resolved

[Q-03] BasePriceAUSD - Reverse token initialization results in unnecessary confusion

Description

The contract initialize `token0` as `KARI` and `token1` as `AUSD`. However, during pool initialization, contract reserves it making `token0` as `AUSD` and `token1` as `KARI` for the pool.

This results in unnecessary confusion and hinder code readability.

Recommendation

Consider using same convention for both contract and the pool

Status

Resolved

[Q-04] BasePriceAUSD# sweep - Wrong naming of return value in sweep

Description

The return value after emptying liquidity caches `amount0`. However, the name of this variable is `liquidityAmount1Anchor` which is wrong.

Recommendation

Consider changing the name to `liquidityAmount0Anchor`.

Status

Resolved

[Q-05] SwapExactInput#calculateCapacity - Reversed input of ticks will result in erroneous capacity calculation

Description

The `SwapExactInput.calculateCapacity()` function retrieves a `Uniswap V3 LP` position from the `INonfungiblePositionManager` and attempts to compute its capacity using `SqrtPriceMath.getAmount1Delta`.
However, the function passes tick values in reversed order.

Recommendation

Fix the destructuring to maintain the correct tick order

Status

Acknowledged

Centralisation

The Hikari Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Hikari Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au